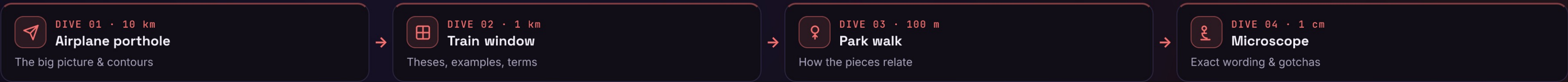




Command

| Encapsulate a request as an object — to parameterize clients, queue or log requests, and support undoable operations.



Airplane porthole

DIVE 01 · ORIENTATION

▼ 10 km

A request turned into a first-class object — so you can pass it around, queue it, log it and undo it like any other value.

THE CONTOURS

• Action as an object
the verb “do X” becomes a thing with an execute()

• Decouples invoker & receiver
the button doesn’t know what the action actually does

• Enables undo, queue, log
because the request is now storable and replayable

USE IT WHEN — GoF applicability

Parameterize objects by an action
to perform — a callback as a first-class object

Queue, schedule, log or undo
operations — the request must outlive the call

Behavioral family: Command reifies a request · Chain passes one along · Strategy swaps an algorithm · Memento saves state for undo. Chains and queues carry Commands.

↓ DIVE DEEPER

Train window

DIVE 02 · KEY OBJECTS

▼ 1 km

THE THREE PARTS

Command
declares execute() (and often undo())

Invoker
holds and triggers a command — knows nothing of the receiver

Receiver
knows how to do the real work the command calls

IN THE WILD

Undo / redo stacks
editors push commands and reverse them later — GoF’s example

Job / task queues
BullMQ & Sidekiq jobs are serialized commands

GUI actions
menu items and buttons fire command objects

DB transactions / WAL
logged commands enable replay and recovery

<> Client builds ConcreteCommand(receiver) → Invoker.run(cmd) → cmd.execute() → receiver does the work.

GoF intent: “Encapsulate a request as an object, letting you parameterize clients with different requests.”

↓ DIVE DEEPER

Park walk

DIVE 03 · CONNECTIONS

▼ 100 m

Client
builds command

→

Invoker
stores & triggers

→

cmd.execute()
(undo() too)

→

✓ Receiver
does the work

WHY IT FOLLOWS

→ Reify the request ⇒ an action becomes a value you can store, pass and replay

→ Invoker holds commands ⇒ it triggers them without knowing the receiver or the logic

→ execute + undo ⇒ every command can carry how to reverse itself

🔗 A Composite of Commands is a macro; Memento captures state for a command’s undo. Chains and queues transport Commands; Prototype can clone them for a log. A plain callback is the degenerate Command.

STRUCTURE · UML

Command

- receiver: Receiver

+ execute(): void

+ undo(): void

ConcreteCommand binds a receiver + arguments; execute() calls receiver actions, undo() reverses them.

WHAT IT BUYS YOU

✓ Decouples invoker & receiver — the trigger is independent of the work done

✓ Commands are first-class — queue, log, schedule and compose them like data

✓ Undo / redo — store executed commands and reverse them

✓ Composite commands — bundle several commands into one macro

✓ Easy to extend — add a new command class without changing the invoker

↓ DIVE DEEPER

Microscope

DIVE 04 · DETAILS

▼ 1 cm

Definition: “Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.”

— GoF, “Design Patterns”, 1994

THE CODE

```
interface Command { execute(); undo() }
class AddText implements Command {
  constructor(private doc, private s) {}
  execute() { this.doc.append(this.s); }
  undo() { this.doc.trim(this.s.length); }
}
const history = [];
function run(c) { c.execute(); history.push(c); }
function undo() { history.pop()?.undo(); }
```

COMPARE THE ALTERNATIVES

Property	Command	Strategy	Callback	Chain
Reifies a request	✓	~	~	✗
Supports undo	✓	✗	✗	✗
Queue / log / replay	✓	✗	~	~
Swaps an algorithm	✗	✓	~	✗
Decouples invoker / receiver	✓	~	~	✓

Command = a stored action (do/undo) · Strategy = a swappable algorithm · Callback = its lightweight form · Chain = routes one.

INTERVIEW PITFALLS

Q “Command vs Strategy?”
Both wrap behavior in an object; Command reifies a request you can queue / undo, Strategy swaps an interchangeable algorithm.

Q “How does undo work?”
Each command stores what it needs to reverse (or a Memento); a history stack pops and calls undo().

Q “Is a job queue Command?”
Yes — a serialized job is a ConcreteCommand executed later by a worker (the invoker).

Q “Command vs CQRS command?”
Same spirit: a command object captures write intent; CQRS routes it to a single handler.

VARIANTS

Undoable command
carries execute() + undo() — editor history

Macro / composite
one command runs a list of commands

Queued / async
serialized and run later by a worker

Logged command
persisted for replay & recovery (WAL, event store)

Function as command
a closure is the lightweight degenerate form

CQRS command
a write-intent message handled by one handler

NUANCES & GOTCHAS

• Undo needs state — store enough (or a Memento) to reverse; recompute isn’t always possible

• Command vs Strategy — Command reifies a request (do/undo); Strategy swaps an algorithm

• Idempotency & replay — logged or queued commands may run twice; design for it

• Where do args live? — bind receiver + parameters at construction, not at execution

• Fat invoker smell — if the invoker knows a command’s internals, the decoupling is lost

• Serialization — queued commands must be serializable; closures usually aren’t

WATCH OUT

 **Anti-pattern:** Skip it when you just need to call a function now — a plain callback is the lightweight command. Reach for full command objects only when you need undo, queuing, logging or composition.

 **Modern take:** job queues (BullMQ), event sourcing, Redux actions and CQRS commands are this pattern at scale — serializable requests handled and replayed by dedicated handlers.

• Vasyl Krupka · Senior Fullstack Engineer · learn-by-diving series

Source: GoF “Design Patterns” (1994)

v1